

UNIVERSIDADE FEDERAL DE SÃO JOÃO DEL-REI

Departamento de Engenharia de Telecomunicações

**PIPELINE MODULAR PARA DETECÇÃO DE
ÁUDIO SINTÉTICO
USANDO ARQUITETURAS NEURAIIS HÍBRIDAS
E
ANÁLISE EMPÍRICA DE CARACTERÍSTICAS**

Thierry Gonçalves Braga

Trabalho de Conclusão de Curso apresentado à Universidade
Federal de São João del-Rei

Orientadora: Prof.^a Ana Cláudia

São João del-Rei — MG

2026

FOLHA DE APROVAÇÃO

Trabalho de Conclusão de Curso defendido e aprovado em _____, como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

Banca Examinadora:

Prof.^a Ana Cláudia (Orientadora)
Universidade Federal de São João del-Rei

Prof. _____
Instituição: _____

Prof. _____
Instituição: _____

RESUMO

Este trabalho apresenta o desenvolvimento e a avaliação de um *pipeline* modular e de código aberto para detecção de áudio sintético. A metodologia integra pré-processamento de áudio, extração de características acústicas, treinamento supervisionado, inferência e geração automática de relatórios de *benchmark*. A versão experimental consolidada avalia quatorze arquiteturas, abrangendo modelos SSL, redes baseadas em forma de onda, grafos, CNNs, Transformers, modelos de fusão e classificadores clássicos. Os experimentos foram conduzidos sobre uma base balanceada com 15.000 amostras de áudio, dividida em 70/15/15 para treino, validação e teste, com janelas padronizadas em 16 kHz, mono e 5 s. O treinamento neural utilizou GPU NVIDIA RTX 3060 via WSL2/CUDA, enquanto SVM e Random Forest foram otimizados por validação cruzada em CPU. No protocolo consolidado, Conformer e Sonic Sleuth¹ obtiveram 100,00% de acurácia e EER de 0,00% no conjunto de teste limpo, seguidos por Hybrid CNN-Transformer (99,96%), MultiscaleCNN (99,73%), SVM (99,02%) e Random Forest (98,18%). O Spectrogram Transformer foi a principal arquitetura com instabilidade observada: alcançou 98,00% de validação na época 11, mas terminou com 71,51% de acurácia no teste, indicando necessidade de restauração rigorosa do melhor *checkpoint* e novo ajuste de hiperparâmetros. Os resultados confirmam a funcionalidade do *pipeline* para comparação padronizada de arquiteturas, mas também evidenciam a necessidade de validação externa, análise de robustez e controle de possíveis vieses de domínio antes de uso em cenários reais.

Palavras-chave: *deepfake* de áudio; *benchmark*; arquiteturas neurais; WavLM; HuBERT; Conformer; MultiscaleCNN; robustez a ruído.

ABSTRACT

This work presents the development and evaluation of a modular open-source pipeline for synthetic audio detection. The methodology integrates audio preprocessing, acoustic feature extraction, supervised training, inference and automatic benchmark report generation. The consolidated experimental version evaluates fourteen architectures, including SSL models, waveform-based networks, graphs, CNNs, Transformers, fusion models and classical classifiers. Experiments were conducted on a balanced 15,000-sample audio dataset with a 70/15/15 train/validation/test split and 16 kHz mono 5 s windows. Neural models were trained on an NVIDIA RTX 3060 GPU through WSL2/CUDA, while SVM and Random Forest were optimized with CPU cross-validation. Under the consolidated protocol, Conformer and Sonic Sleuth achieved 100.00% accuracy and 0.00% EER on the clean test set, followed by Hybrid CNN-Transformer (99.96%), MultiscaleCNN (99.73%), SVM (99.02%) and Random Forest (98.18%). The Spectrogram Transformer was the main architecture with observed instability: it reached 98.00% validation accuracy at epoch 11 but ended with 71.51% test accuracy, indicating the need for strict best-checkpoint restoration and further hyperparameter tuning. The results confirm the pipeline's ability to

¹Sonic Sleuth é o nome interno dado pelo autor ao modelo híbrido de características clássicas (LFCC, MFCC e CQT) com rede neural compacta desenvolvido neste trabalho.

compare heterogeneous architectures in a standardized way, while also showing the need for external validation, robustness analysis and domain-bias control before deployment in real-world scenarios.

Keywords: audio deepfake; benchmark; neural architectures; WavLM; HuBERT; Conformer; MultiscaleCNN; noise robustness.

Lista de Figuras

Lista de Tabelas

Lista de Abreviaturas e Siglas

AGC	<i>Automatic Gain Control</i> (Controle Automático de Ganho)
AASIST	<i>Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks</i>
AST	<i>Audio Spectrogram Transformer</i>
AUC-ROC	<i>Area Under the Receiver Operating Characteristic Curve</i>
AWGN	<i>Additive White Gaussian Noise</i> (Ruído Gaussiano Branco Aditivo)
CNN	<i>Convolutional Neural Network</i> (Rede Neural Convolucional)
CPU	<i>Central Processing Unit</i>
CQT	<i>Constant-Q Transform</i>
CQCC	<i>Constant-Q Cepstral Coefficients</i>
EER	<i>Equal Error Rate</i> (Taxa de Erro Igual)
FFT	<i>Fast Fourier Transform</i> (Transformada Rápida de Fourier)
F0	Frequência Fundamental
GAT	<i>Graph Attention Network</i>
GMM-UBM	<i>Gaussian Mixture Model – Universal Background Model</i>
GPU	<i>Graphics Processing Unit</i>
HAG	<i>Heterogeneous Attention Graph</i>
HNR	<i>Harmonic-to-Noise Ratio</i>
LCNN	<i>Light CNN</i>
LFCC	<i>Linear Frequency Cepstral Coefficients</i>
LSTM	<i>Long Short-Term Memory</i>
LUFS	<i>Loudness Units relative to Full Scale</i>
MFCC	<i>Mel-Frequency Cepstral Coefficients</i> (Coeficientes Cepstrais na Escala Mel)
MLP	<i>Multilayer Perceptron</i>
PCM	<i>Pulse-Code Modulation</i>
RNN	<i>Recurrent Neural Network</i>
RVC	<i>Retrieval-based Voice Conversion</i>
SHAP	<i>SHapley Additive exPlanations</i>
SNR	<i>Signal-to-Noise Ratio</i> (Relação Sinal-Ruído)
SSL	<i>Self-Supervised Learning</i> (Aprendizado Auto-Supervisionado)
SVM	<i>Support Vector Machine</i>
STFT	<i>Short-Time Fourier Transform</i>
t-DCF	<i>Tandem Detection Cost Function</i>
TTS	<i>Text-to-Speech</i> (Síntese de Fala)
VAD	<i>Voice Activity Detection</i> (Detecção de Atividade de Voz)
VC	<i>Voice Conversion</i> (Conversão de Voz)
ZCR	<i>Zero-Crossing Rate</i> (Taxa de Cruzamento por Zero)

Sumário

1 Introdução

A crescente democratização de tecnologias de síntese de áudio baseadas em inteligência artificial tem facilitado a criação de *deepfakes* de áudio com qualidade cada vez mais próxima à voz humana natural [?,?]. Técnicas modernas como RVC (*Retrieval-based Voice Conversion*) [?], Tacotron 2 [?] e WaveNet [?] permitem a geração de conteúdo sintético realista com apenas alguns minutos de áudio original.

Esta evolução tecnológica, embora benéfica para aplicações legítimas como assistentes virtuais e dublagem automatizada, tem sido explorada maliciosamente em fraudes financeiras, manipulação política e chantagem [?,?]. No contexto brasileiro, foram documentados 78 casos de conteúdo sintético relacionado a candidatos durante as eleições municipais de 2024 [?], evidenciando a necessidade urgente de sistemas de detecção robustos voltados para o português brasileiro.

Estudos perceptuais indicam que humanos identificam corretamente *deepfakes* de áudio em apenas 73% dos casos, taxa que decresce para 51% com áudios de alta qualidade [?]. Isso evidencia a necessidade de sistemas automatizados robustos e explica o surgimento dos *benchmarks* padronizados ASVspoof como eixo central de avaliação [?, ?, ?].

Uso dual e responsabilidade científica

Os resultados apresentados neste trabalho descrevem quais características acústicas são mais discriminativas para detecção. Esse conhecimento, publicado de forma aberta, pode ser usado para calibrar sistemas de síntese mais difíceis de detectar. O autor e a orientadora reconhecem esse risco e recomendam que qualquer extensão ou implantação do *pipeline* seja precedida de avaliação ética formal e, em contextos sensíveis, revisão humana obrigatória das decisões automatizadas.

1.1 Objetivos

1.1.1 Objetivo Geral

Desenvolver um *pipeline* modular, de código aberto e extensível para detecção de áudio sintético que integre múltiplas arquiteturas neurais especializadas e permita análise empírica sistemática de características acústicas.

1.1.2 Objetivos Específicos

- Realizar uma pesquisa sistematizada das gerações de métodos de detecção de áudio sintético.
- Implementar pré-processamento com VAD e AGC.
- Ranquear características acústicas por capacidade discriminativa, ancorado na literatura consolidada.

- Implementar e comparar quatorze arquiteturas especializadas sobre uma base experimental padronizada.
- Avaliar robustez sob ruído AWGN com múltiplos níveis de SNR.
- Propor um *pipeline* reproduzível com tecnologias de código aberto.

2 Fundamentação Teórica

2.1 Tecnologias de Síntese de Voz

2.1.1 Clonagem de Voz

A clonagem de voz (*voice cloning*) consiste em criar um modelo capaz de replicar as características acústicas e prosódicas de um falante específico a partir de amostras limitadas [?]. Modelos como VALL-E [?] utilizam abordagem *few-shot* baseada em codificadores neurais de áudio, atingindo qualidade quase indistinguível com apenas 3 segundos de áudio. O processo pode ser formalizado como:

$$\mathbf{y}_{clone} = f_{\theta}(\mathbf{x}_{text}, \mathbf{z}_{speaker}) \quad (1)$$

onde $\mathbf{y}_{clone}$ é o áudio sintético gerado, $\mathbf{x}_{text}$ é o texto de entrada, $\mathbf{z}_{speaker}$ é o vetor de características do falante e f_{θ} é o modelo neural treinado.

2.1.2 Conversão de Voz

A conversão de voz (*voice conversion*, VC) transforma a voz de um falante para soar como a de outro, preservando o conteúdo linguístico original [?]. A técnica RVC emprega bancos de dados de vozes e extratores de características para mapear o falante de origem ao alvo:

$$\mathbf{y}_{conv} = g_{\phi}(\mathbf{x}_{source}, \mathbf{z}_{target}) \quad (2)$$

Modelos como StarGAN-VC [?] utilizam redes adversariais para aprender mapeamentos muitos-para-muitos entre vozes com latência reduzida. A VC é desafiadora para detecção, pois preserva o conteúdo linguístico, mascarando artefatos e levando detectores baseados em características espectrais a EERs de até 15% [?].

Conversão de Voz nas Eleições Brasileiras de 2024

Durante as eleições municipais brasileiras de 2024, a conversão de voz foi identificada em casos de manipulação de discursos de candidatos, onde trechos de áudio foram alterados para simular falas comprometedoras [?].

2.1.3 Síntese de Fala (*Text-to-Speech*)

O Tacotron 2 [?] estabeleceu arquiteturas *sequence-to-sequence* para mapear texto em espectrogramas mel, convertidos em áudio por um vocoder como WaveNet [?]:

$$\mathbf{s}_{mel} = h_{\psi}(\mathbf{t}_{text}), \quad \mathbf{y}_{audio} = v_{\omega}(\mathbf{s}_{mel}) \quad (3)$$

onde $\mathbf{t}_{text}$ é o texto de entrada, $\mathbf{s}_{mel}$ é o espectrograma mel intermediário, $\mathbf{y}_{audio}$ é a forma de onda sintetizada, h_{ψ} representa o modelo *sequence-to-sequence* (Tacotron 2) e v_{ω} representa o vocoder neural (WaveNet).

A WaveNet modela diretamente a forma de onda bruta, alcançando qualidade próxima à humana [?]. Sistemas TTS avançados dificultam a detecção por métodos tradicionais baseados em MFCCs devido à alta naturalidade dos áudios gerados.

2.1.4 Manipulação em Tempo Real

A manipulação em tempo real permite alterar áudios durante conversas ao vivo com latência inferior a 200 ms, tornando-a imperceptível para humanos [?]. Sistemas como RVC v2 otimizam a conversão para baixa latência utilizando redes convolucionais 1D leves [?]:

$$\mathbf{y}_{rt} = r_{\xi}(\mathbf{x}_{input}, \mathbf{z}_{target}, t) \quad (4)$$

Riscos do *vishing* em Tempo Real

Casos de *vishing* em 2025 usaram manipulação em tempo real para imitar vozes de familiares, enganando vítimas em transferências financeiras de emergência [?]. A latência média desses sistemas é inferior a 200 ms.

2.2 Gerações de Métodos de Detecção

2.2.1 Primeira Geração — Características Manuais e Estatística Clássica

Até o ASVspoof 2015 [?], o padrão envolvia extração manual de MFCC, LFCC e CQCC seguida por classificadores GMM-UBM e SVM. Sahidullah et al. [?] demonstraram que o LFCC supera o MFCC como *front-end* para detecção de *spoofing*, enquanto Todisco et al. [?] introduziram o CQCC como característica de referência para a área. Embora computacionalmente leves, essas arquiteturas falham gravemente com métodos de síntese neural modernos por não generalizarem entre bases de dados (EER > 20% em cenários *cross-dataset*).

2.2.2 Segunda Geração — CNNs e RNNs em Espectrogramas

A partir do ASVspoof 2019 [?], CNNs (como ResNet e Res2Net [?]) e LSTMs processando espectrogramas mel tornaram-se dominantes. Sistemas como o LCNN (*Light CNN*) reduziram drasticamente os EERs ao explorar artefatos visuais de alta frequência deixados por *vocoders*. Abordagens híbridas CNN e CNN-RNN sobre espectrogramas reportam acurácias superiores a 90% na detecção de áudio sintético [?].

2.2.3 Terceira Geração — Análise Direta de Forma de Onda e Grafos

A introdução de RawNet2 [?] e AASIST [?] marcou o processamento *end-to-end*. O RawNet2 utiliza *Sinc-Convolutions* para processar diretamente a forma de onda PCM 1D. O AASIST implementa Redes de Atenção em Grafos (GATs) para modelar dependências temporais e espectrais simultaneamente, alcançando EER inferior a 1% em bases controladas.

Requisitos Computacionais da 3ª Geração

Modelos *raw-audio* como RawNet2 e AASIST exigem GPU dedicada e bases suficientemente amplas para convergir de forma estável. Em cenários restritos de dados ou sem aceleração por GPU, a literatura reporta maior dificuldade de otimização para modelos desse tipo [?, ?].

2.2.4 Quarta Geração — Modelos *Foundation* e SSL

Com o ASVspoof 5 [?] e avaliações recentes de modelos auto-supervisionados, o padrão ouro consolidou-se em torno de WavLM [?] e HuBERT [?]. Pré-treinados em dezenas de milhares de horas de áudio não rotulado, esses modelos, herdeiros do paradigma de representação auto-supervisionada estabelecido pelo wav2vec 2.0 [?], são ajustados (*fine-tuned*) para *anti-spoofing*, oferecendo resistência superior a ataques adversariais e dados *crowdsourced*. O Audio Spectrogram Transformer (AST) [?] emerge como alternativa promissora, processando espectrogramas como sequências de *patches* sem convoluções.

A Tabela ?? consolida as quatro gerações de métodos.

Tabela 1: *Survey* das gerações de métodos de detecção de áudio sintético.

Geração	Representação	Classificador	Pontos Fortes	Limitações
1ª (até 2015)	MFCC, LFCC, CQCC	GMM-UBM, SVM	Leveza computacional	Baixa generalização
2ª (2019)	Espectrograma mel	CNN, LSTM, LCNN	Robustez a artefactos de vocoder	Sensível a geradores não vistos
3ª (2021)	Forma de onda bruta, grafos	RawNet2, AA-SIST	EER < 1% controlo	GPU + >5.000 amostras
4ª (2024+)	Representação SSL	WavLM, HuBERT + MLP	Generalização <i>cross-dataset</i>	Dependência de GPUs potentes

3 Metodologia

3.1 Visão Geral do Pipeline

O sistema proposto opera em duas fases: **treinamento** e **predição**. Durante o treinamento, o modelo aprende a distinguir características de voz real e sintética. Na predição, processa novos áudios e fornece classificação binária (Real/Fake) com *score* de confiança.

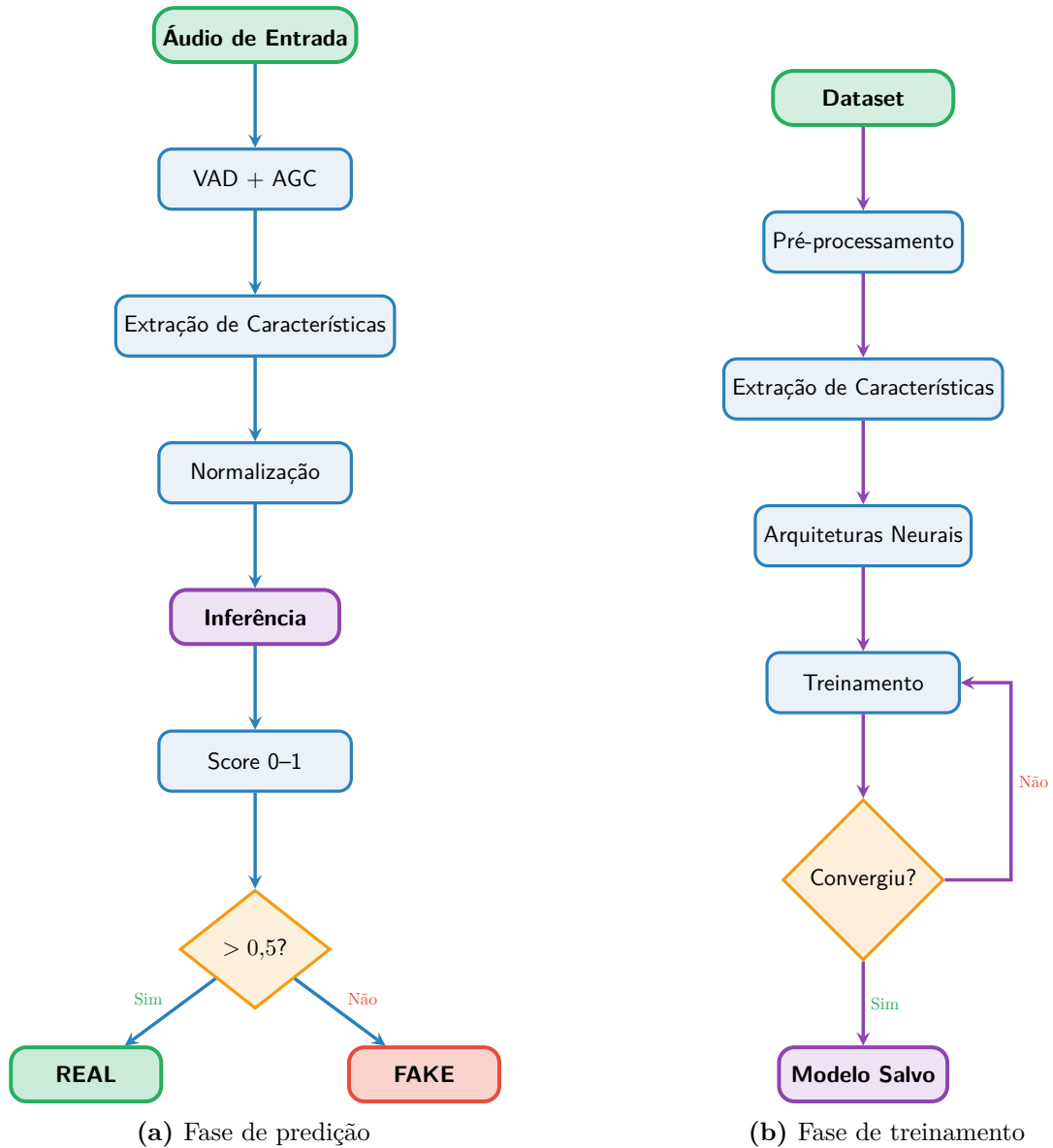


Figura 1: Diagrama do *pipeline* proposto. O modelo treinado (??) é implantado (*deploy*) na fase de inferência (??).

3.2 Dataset e Protocolo Experimental

3.2.1 Construção do Dataset BRSpeech-DF

O conjunto de dados experimental utilizado no *benchmark* consolidado foi estruturado como uma base balanceada de 15.000 amostras de áudio, sendo 7.500 amostras *bonafide* e 7.500 amostras *spoof*. Esse volume foi adotado para treinar arquiteturas de maior capacidade, inclusive modelos *raw-audio* e *self-supervised learning* (SSL), sob um protocolo único de avaliação.²

²O rótulo *BRSpeech-DF* é um identificador interno de projeto que designa a aplicação-alvo (detecção de *deepfakes* de voz no contexto brasileiro). Ele *não* implica composição majoritariamente em português brasileiro: a proporção por idioma ainda não foi consolidada no manifesto (ver Seção ??), e essa verificação é pré-requisito para qualquer afirmação sobre cobertura linguística.

- **Origem e rastreabilidade:** os arquivos foram organizados a partir de diretórios rotulados como *real* e *fake* e exportados para um arquivo NumPy único. O manifesto do pipeline registra caminho de origem, parâmetros de áudio, contagens por classe, semente e partições.
- **Metadados pendentes:** número de falantes distintos, distribuição por gênero e idioma (em especial, proporção de amostras em português brasileiro), licença de cada fonte, duração original dos arquivos e verificação explícita de duplicatas perceptuais ainda não foram consolidados no manifesto. Por esse motivo, os resultados são tratados como avaliação controlada do *pipeline* e *não substituem* validação externa em bases públicas como ASVspoof, WaveFake e In-the-Wild.
- **Padronização:** todos os arquivos foram convertidos para 16 kHz, mono, janelas de 5 s e representação PCM normalizada. Para os modelos de áudio bruto, a entrada final possui dimensão 80.000×1 ; para modelos espectrais, o mesmo áudio é convertido para representações Mel, LFCC, CQT ou MFCC conforme a arquitetura.
- **Balanceamento:** as classes real e falsa foram mantidas em proporção 1:1 em todas as partições, evitando viés de decisão por frequência de classe.
- **Divisão estratificada:** 70% para treino (10.500 amostras), 15% para validação (2.250 amostras) e 15% para teste (2.250 amostras), com semente fixa 42. *Importante:* a divisão atual estratifica por classe, mas não garante separação por falante ou por origem do gerador. Caso amostras do mesmo falante ou do mesmo sistema TTS/VC apareçam em partições diferentes, modelos podem aprender artefatos de origem em vez de propriedades gerais de áudio sintético, inflando as métricas. Quando os metadados de falante e fonte estiverem completos, a divisão deve evoluir para separação explícita por origem.
- **Robustez:** além do teste limpo, foram geradas versões com ruído AWGN em SNR de 30 dB, 20 dB e 10 dB para medir degradação em cenários de transmissão e captura degradadas.

A Tabela ?? consolida os principais atributos da base experimental utilizada no *benchmark*.

Tabela 2: Resumo da base experimental BRSpeech-DF usada no *benchmark*.

Atributo	Valor consolidado
Total de amostras	15.000
Amostras <i>bonafide</i>	7.500
Amostras <i>spoof</i>	7.500
Treino / validação / teste	10.500 / 2.250 / 2.250
Balanceamento do teste	1.125 reais + 1.125 falsas
Taxa de amostragem	16 kHz
Duração padronizada	5 s
Formato bruto	PCM mono normalizado, 80.000×1
Semente de particionamento	42
Testes de ruído	AWGN em 30 dB, 20 dB e 10 dB
Separação por falante	<i>Pendente</i> — ver Seção ??
Idioma	Não documentado no manifesto atual

Os artefatos de entrada do *benchmark* foram consolidados no arquivo `benchmark_audio_raw_balanced_15k.npz`, localizado em `app/datasets/`. Os resultados, modelos treinados, métricas, matrizes de confusão e figuras são salvos em pastas individuais por arquitetura dentro de `results/`.

3.2.2 Protocolo de Treinamento

Os modelos neurais foram treinados com o preset de *benchmark* de 100 épocas, semente 42, divisão estratificada e salvamento de melhor *checkpoint*. O *batch size* padrão foi 32, reduzido para 16 em arquiteturas com maior consumo de VRAM e para 1 nos modelos SSL originais quando necessário. Precisão mista (`mixed_float16`) foi habilitada apenas para os modelos Keras compatíveis: Conformer, Hybrid CNN-Transformer, MultiscaleCNN, EfficientNet-LSTM e Ensemble; os modelos SSL originais utilizaram PyTorch/CUDA em precisão padrão (`float32`).

Para interpretação das métricas, adotou-se acurácia e F1 como medidas de desempenho global, AUC-ROC como separabilidade probabilística e EER como taxa de equilíbrio entre falso aceite e falsa rejeição. Como o conjunto de teste possui 2.250 amostras, resultados de 100,00% não devem ser interpretados como erro populacional nulo; pelo intervalo de Wilson, zero erro no teste ainda implica limite inferior aproximado de 99,83% para a acurácia verdadeira a 95% de confiança.

3.3 Etapa 1: Pré-processamento

O sistema suporta múltiplos formatos (WAV, MP3, FLAC, OGG) com conversão automática para padrões consistentes: taxa de amostragem de 16 kHz, mono, 16 bits PCM linear.

3.3.1 Normalização AGC

A normalização implementa controle automático de ganho adaptativo (Equação ??):

$$x_{norm}[n] = x[n] \cdot \frac{L_{target}}{\sqrt{\frac{1}{N} \sum_{i=0}^{N-1} x[i]^2}} \quad (5)$$

onde L_{target} é o nível alvo de *loudness*, fixado em -23 LUFS.

3.3.2 Detecção de Atividade de Voz (VAD)

O VAD baseado em energia remove segmentos silenciosos pelas Equações ?? e ??:

$$E[m] = 10 \log_{10} \left(\frac{1}{N} \sum_{n=0}^{N-1} |x[mH + n]|^2 \right) \quad (6)$$

$$\text{Voice}[m] = \begin{cases} 1, & \text{se } E[m] > \theta_{VAD} \text{ e } D[m] > D_{min} \\ 0, & \text{caso contrário} \end{cases} \quad (7)$$

onde θ_{VAD} é o limiar de energia, fixado em -25 dB; H é o *hop length* ($H = 256$ amostras); $D[m]$ é a duração do segmento; e $D_{min} = 0,3$ s.

VAD em Ambiente de Produção

O Silero VAD (disponibilizado via Torch Hub) oferece performance pré-treinada superior ao VAD por energia. Nos experimentos, a indisponibilidade do `torchaudio` motivou o uso do VAD por energia como *fallback*, preservando a funcionalidade do pipeline sem dependências externas não atendidas no ambiente de CPU.

3.4 Etapa 2: Extração e Análise de Características

O sistema extrai características acústicas complementares cuja hierarquia discriminativa é suportada pela literatura consolidada de *anti-spoofing* [?, ?, ?].

3.4.1 Características Espectrais

Centroide Espectral (Equação ??):

$$SC[m] = \frac{\sum_{k=0}^{K-1} k \cdot |X[m, k]|^2}{\sum_{k=0}^{K-1} |X[m, k]|^2} \quad (8)$$

Rolloff Espectral (Equação ??): frequência abaixo da qual está 85% da energia

$$SR[m] = k_r \text{ tal que } \sum_{k=0}^{k_r} |X[m, k]|^2 = 0,85 \sum_{k=0}^{K-1} |X[m, k]|^2 \quad (9)$$

Largura de Banda Espectral (Equação ??):

$$SB[m] = \sqrt{\frac{\sum_{k=0}^{K-1} (k - SC[m])^2 \cdot |X[m, k]|^2}{\sum_{k=0}^{K-1} |X[m, k]|^2}} \quad (10)$$

Taxa de Cruzamento Zero (ZCR) (Equação ??):

$$ZCR[m] = \frac{1}{2N} \sum_{n=0}^{N-1} |\text{sgn}(x[mH + n]) - \text{sgn}(x[mH + n - 1])| \quad (11)$$

Contraste Espectral (Equação ??):

$$SContrast[m] = \log\left(\frac{\mu_p}{\mu_v}\right) \quad (12)$$

onde μ_p e μ_v são as médias dos picos e vales espectrais.

Planura Espectral (Equação ??):

$$SFlatness[m] = \frac{\left(\prod_{k=0}^{K-1} |X[m, k]|^2\right)^{1/K}}{\frac{1}{K} \sum_{k=0}^{K-1} |X[m, k]|^2} \quad (13)$$

3.4.2 Coeficientes Cepstrais na Escala Mel (MFCC)

$$MFCC[m, c] = \sum_{k=0}^{M-1} \log(S[m, k]) \cdot \cos\left(\frac{\pi c(k + 0,5)}{M}\right) \quad (14)$$

onde $S[m, k]$ são as energias dos filtros mel, $M = 40$ filtros, coeficientes: 40, pré-ênfase $\alpha = 0,97$, faixa 80 Hz a 8000 Hz.

3.4.3 Espectrogramas na Escala Mel

$$MelSpec[m, k] = \log\left(\sum_n H_k[n] \cdot |STFT[m, n]|^2 + \varepsilon\right) \quad (15)$$

Configurações: 80 bandas mel, janela Hamming de 1024 amostras, *hop length* 256, tamanho da FFT de 2048 pontos.

3.4.4 Características Prosódicas

Frequência Fundamental (F0): estimada pelo CREPE, uma rede neural convolucional profunda treinada para rastreamento de *pitch* diretamente sobre a forma de onda (distinta de métodos clássicos por autocorrelação, como o YIN).

Jitter (variação de período ciclo a ciclo, Equação ??):

$$Jitter = \frac{1}{N-1} \sum_{i=1}^{N-1} \frac{|T_i - T_{i-1}|}{\bar{T}} \quad (16)$$

Shimmer (variação de amplitude ciclo a ciclo, Equação ??):

$$Shimmer = \frac{1}{N-1} \sum_{i=1}^{N-1} \frac{|A_i - A_{i-1}|}{\bar{A}} \quad (17)$$

HNR (*Harmonic-to-Noise Ratio*, Equação ??):

$$HNR = 10 \log_{10} \left(\frac{P_{harmonic}}{P_{noise}} \right) \quad (18)$$

3.4.5 Constant-Q Transform (CQT)

$$CQT[m, k] = \sum_n x[n] \cdot g_k[n - mH] \cdot e^{-j2\pi Q_k n / N_k} \quad (19)$$

O CQCC derivado do CQT foi estabelecido como *front-end* de referência para *anti-spoofing* por Todisco et al. [?]. Configurações: 12 *bins* por oitava, 7 oitavas (84 características), $f_{min} = 65$ Hz.

3.4.6 Características Delta e Delta-Delta

$$\Delta X[m, c] = \frac{\sum_{n=1}^N n(X[m+n, c] - X[m-n, c])}{2 \sum_{n=1}^N n^2} \quad (20)$$

3.5 Ranking de Características por Desempenho Discriminativo

A escolha das características acústicas é ancorada na literatura de *anti-spoofing*. Características espectrais apresentam consistentemente maior poder discriminativo em relação a MFCCs clássicos, conforme demonstrado em estudos sistemáticos conduzidos nas edições do ASVspoof [?, ?, ?]. Características prosódicas, embora complementares, exibem desempenho inferior quando utilizadas isoladamente, resultado consistente com os achados do ADD Challenge [?]. A Tabela ?? apresenta um resumo das características descritas nesta

seção, organizado por tipo, dimensionalidade do vetor extraído, custo computacional e validação de cada conjunto na literatura.

Tabela 3: Ranking de características acústicas: síntese da literatura. **Custo Computacional:** estimativa qualitativa (Baixo/Médio/Alto) do consumo de tempo e memória para extração no *pipeline*.

Tipo		Dimen.	Custo Comp.	Validação na Literatura
Espectrais		14	Médio	Maior poder discriminativo em detecção de artefatos de <i>vocoder</i> [?, ?]
Combinação otimizada		60	Alto	Fusão de domínios reduz EER em cenários <i>cross-dataset</i> [?]
MFCC		40	Baixo	Referência clássica; LFCC supera MFCC em <i>spoofing</i> desde 2015 [?, ?]
Mel	Spectrogram	80	Baixo	Base para CNNs; captura artefatos visuais de alta frequência [?, ?]
CQT		84	Alto	Resolução logarítmica; CQCC como <i>front-end</i> robusto [?]
Prosódicas		6	Médio	Desempenho isolado inferior; fusão com espectrais melhora resultados [?, ?]

3.6 Normalização Estatística

Min-Max Scaling (Equação ??):

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (21)$$

Z-Score (Equação ??):

$$X_{norm} = \frac{X - \mu}{\sigma} \quad (22)$$

4 Arquiteturas de *Deep Learning*

O *pipeline* consolidado contempla quatorze arquiteturas, cobrindo desde classificadores clássicos sobre características tabulares até modelos neurais *end-to-end* baseados em forma de onda, grafos, convoluções, atenção e *self-supervised learning*. Essa seleção permite comparar custo computacional, estabilidade de treinamento, robustez a ruído e desempenho

final sob o mesmo benchmark.

4.1 Modelos SSL e *Raw-Audio*

O WavLM original [?] e o HuBERT original [?] foram avaliados com *backbones* reais, não apenas com implementações simplificadas. Ambos recebem áudio bruto a 16 kHz e usam representações aprendidas em larga escala, com cabeça supervisionada para a tarefa binária real/*fake*. No *benchmark*, os *backbones* foram executados em PyTorch/CUDA e persistidos localmente para inferência.

O RawNet2 [?] processa diretamente a forma de onda por meio de filtros SincNet, blocos residuais com *feature map scaling* e GRU, preservando informação temporal fina sem necessidade de espectrograma. AASIST [?] e RawGAT-ST [?] exploram atenção em grafos espectro-temporais para modelar relações locais e globais entre regiões do sinal. Esses modelos foram avaliados no protocolo consolidado descrito na Seção ??.

4.2 Modelos Espectrais Neurais

Sonic Sleuth³ combina características acústicas clássicas com uma rede neural compacta. Conformer [?] une convoluções locais e autoatenção para capturar padrões espectro-temporais em diferentes escalas. O Hybrid CNN-Transformer, seguindo a linha de modelos híbridos convolução-atenção para áudio [?], utiliza tokenização convolucional antes dos blocos Transformer, reduzindo o custo em comparação com atenção aplicada diretamente a mapas espectrais densos.

O Spectrogram Transformer avaliado neste trabalho é uma adaptação própria que aplica a lógica de *Vision Transformer*/AST [?] a espectrogramas, distinta da implementação original do AST. O EfficientNet-LSTM usa transferência de aprendizado convolucional seguida por Bi-LSTM para resumir dependências temporais. A MultiscaleCNN incorpora blocos multi-escala inspirados em Res2Net [?], enquanto o Ensemble funde quatro ramos de características complementares.

4.3 Modelos Clássicos

SVM e Random Forest foram mantidos como baselines fortes e interpretáveis. O SVM utiliza padronização estatística e kernel RBF; a Random Forest explora paralelismo em CPU com múltiplas árvores e `n_jobs=-1`. Ambos foram submetidos a busca de hiperparâmetros antes do ajuste final, permitindo comparação justa com as arquiteturas neurais.

4.4 Configuração de Treinamento

³Nome interno criado pelo autor para o classificador híbrido que combina características clássicas (LFCC, MFCC e CQT) com uma rede neural compacta desenvolvida no escopo deste trabalho.

4.4.1 Função de Perda

Binary Cross-Entropy com balanceamento de classes (Equação ??):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [w_1 y_i \log(\hat{y}_i) + w_0 (1 - y_i) \log(1 - \hat{y}_i)] \quad (23)$$

4.4.2 Regularização

Regularização L2 com $\lambda = 0,0001$, *Early Stopping* com paciência de 10 épocas, *ReduceLROnPlateau* com paciência de 5 épocas. A paciência uniforme de 10 épocas foi mantida para todas as arquiteturas para garantir comparação justa; a instabilidade do Spectrogram Transformer (Seção ??) é tratada como diagnóstico de hiperparâmetros, não como limitação do critério de parada.

4.4.3 Otimizador

Otimizador Adam (*Adaptive Moment Estimation*) [?] com $\alpha = 0,001$, $\beta_1 = 0,9$, $\beta_2 = 0,999$. O Adam combina os benefícios das técnicas AdaGrad (adaptação por parâmetro) e RMSProp (média móvel exponencial dos gradientes ao quadrado), sendo amplamente adotado no treinamento de redes neurais profundas por sua robustez a hiperparâmetros e boa convergência em superfícies de perda não-convexas.

5 Ferramentas de Software Livre

- **Pré-processamento e VAD:** Silero VAD (via Torch Hub) com *fallback* por energia para ambientes sem torchaudio. Para AGC e reamostragem, **librosa** provê operações eficientes em CPU.
- **Extração de Características:** **librosa** e **essentia** para extração de características espectrais, MFCC e prosódicas.
- **Modelos *Foundation*:** Disponíveis via **transformers** (Hugging Face). Exigem GPU para *fine-tuning* eficiente.
- **Arquiteturas Especializadas:** RawNet2 e AASIST com implementações oficiais no GitHub sob licença MIT.
- **Ensemble e Classificação:** TensorFlow/Keras para implementação dos modelos e fusão adaptativa.
- **Interpretabilidade (proposta):** **shap** para valores SHAP [?] e **captum** (PyTorch) para GradCAM — integração planejada como extensão futura (ver Seção ??).

6 Experimentos e Resultados

6.1 Ambiente Experimental e Hardware

O *benchmark* consolidado foi executado em Windows 11 com WSL2 e acesso à GPU NVIDIA via CUDA. Os relatórios automáticos registraram Linux 6.6.114.1-microsoft-standard-WSL2, Python 3.11.15 e TensorFlow 2.21.0 nas execuções Keras. A GPU principal foi uma NVIDIA GeForce RTX 3060, capacidade computacional 8.6, com *memory growth*, `cuda_malloc_async` e precisão mista (`mixed_float16`) para modelos compatíveis (ver Seção ??). WavLM e HuBERT originais utilizaram PyTorch/CUDA por serem *backbones* nativos do ecossistema Transformers. SVM e Random Forest foram executados em CPU com paralelismo e validação cruzada. A Tabela ?? resume o ambiente de execução.

Tabela 4: Resumo do ambiente de execução do *benchmark*.

Componente	Configuração
Sistema operacional	Windows 11 + WSL2
Kernel registrado	Linux 6.6.114.1-microsoft-standard-WSL2
Python	3.11.15 (execuções GPU)
TensorFlow	2.21.0 (execuções Keras)
GPU	NVIDIA GeForce RTX 3060, CC 8.6, Tensor Cores/FP16
Aceleração Keras	CUDA, precisão mista (<code>mixed_float16</code>)
Modelos SSL originais	PyTorch/CUDA + <i>backbone</i> WavLM/HuBERT real
Modelos clássicos	CPU, validação cruzada e paralelismo

6.2 Reprodutibilidade do Benchmark

A execução foi organizada para permitir repetição do experimento completo e de modelos individuais. O protocolo usa base consolidada única, semente fixa, divisão estratificada, 100 épocas para modelos neurais e validação cruzada para SVM/Random Forest. Cada arquitetura grava artefatos em diretório próprio, permitindo auditar o modelo salvo, as configurações, o histórico de treino, as métricas e as figuras correspondentes. A Tabela ?? resume os parâmetros de reprodutibilidade do experimento.

Tabela 5: Parâmetros de reprodutibilidade do benchmark consolidado.

Item	Valor registrado
Dataset	app/datasets/benchmark_audio_raw_balanced_15k.npz
Amostras	15.000, balanceadas entre real e sintético
Teste	2.250 amostras, divisão estratificada
Semente	42
Treino neural	100 épocas, melhor checkpoint salvo
Clássicos	GridSearchCV + ajuste final
Resultados	results/<run>/
Modelos padrão	app/models/
Relatório	results/<run>/tcc_report.md

6.3 Resultados no Conjunto de Teste Limpo

A Tabela ?? apresenta os resultados consolidados das quatorze arquiteturas no conjunto de teste limpo. As Figuras ?? e ?? resumem, respectivamente, acurácia e AUC-ROC e a taxa de erro igual (EER) por arquitetura.

Tabela 6: Resultados consolidados no conjunto de teste limpo.

Rank	Modelo	Acurácia	EER	AUC	F1	Lat. ms	Treino	Status
1	Conformer	100,00%	0,00%	1,000	100,00%	49,22	100	OK
1	Sonic Sleuth	100,00%	0,00%	1,000	100,00%	30,86	100	OK
3	Hybrid CNN-Transformer	99,96%	0,00%	1,000	99,96%	33,69	100	OK
4	MultiscaleCNN	99,73%	0,18%	1,000	99,73%	60,87	100	OK
5	SVM	99,02%	0,98%	0,999	99,02%	0,20	CV+fit	OK
6	Random Forest	98,18%	1,91%	0,999	98,18%	26,75	CV+fit	OK
7	RawNet2	96,36%	3,56%	0,996	96,34%	56,28	100	OK
8	Ensemble	95,82%	2,80%	0,998	95,88%	46,55	100	OK
9	RawGAT-ST	95,29%	4,62%	0,984	95,34%	49,61	100	OK
10	AASIST	93,64%	6,36%	0,937	93,53%	47,77	100	OK
11	HuBERT Original	92,71%	7,20%	0,981	92,81%	21,17	100	OK
12	EfficientNet-LSTM	91,16%	9,07%	0,972	91,37%	62,56	100	OK
13	WavLM Original	86,36%	13,64%	0,947	87,13%	10,18	100	OK
14	Spectrogram Transformer	71,51%	28,67%	0,807	71,01%	50,13	100	Tuning

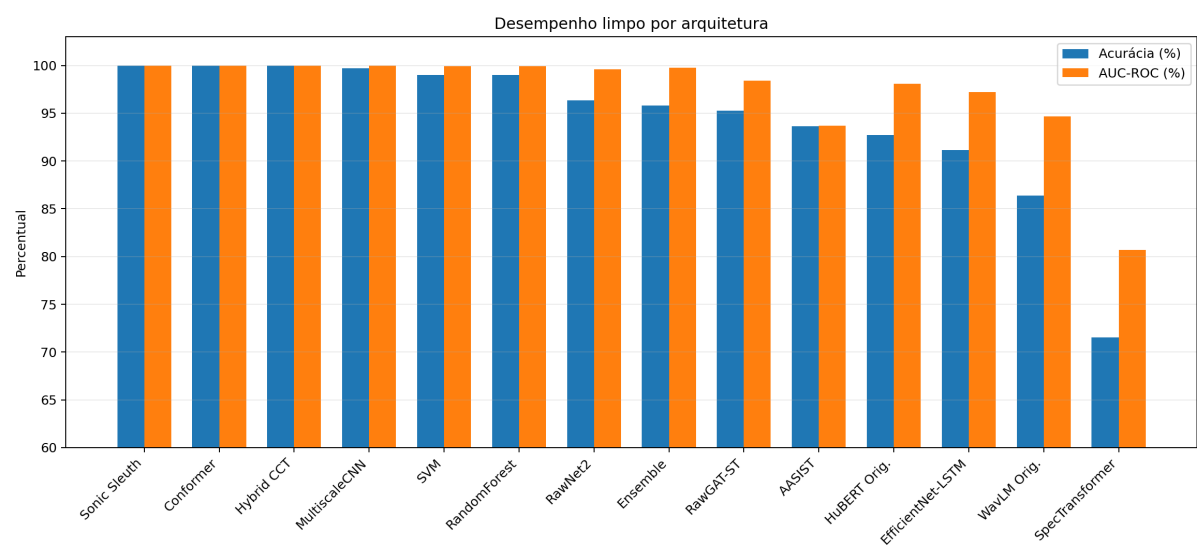


Figura 2: Acurácia e AUC-ROC consolidadas por arquitetura.

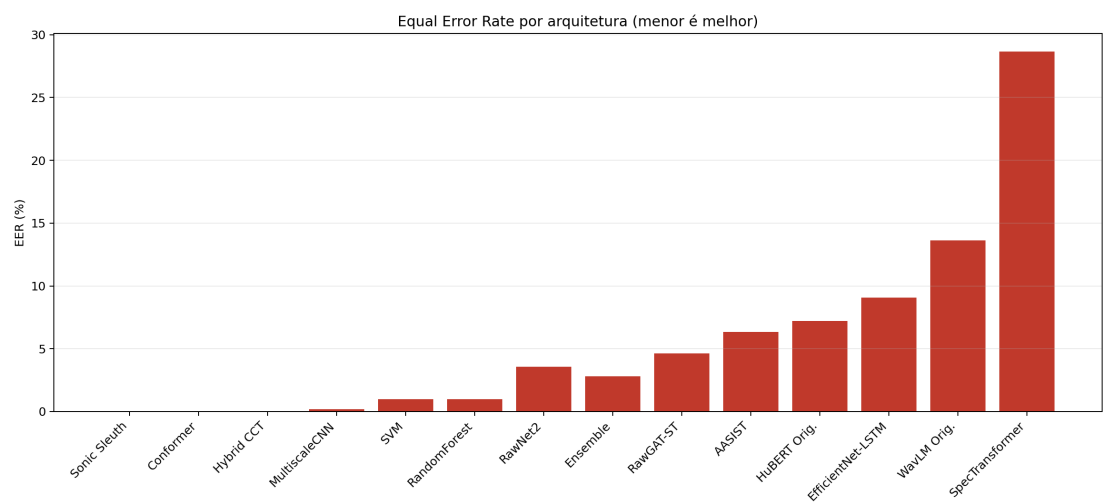


Figura 3: Equal Error Rate (EER) por arquitetura. Valores menores indicam melhor separação entre classes.

A Tabela ?? apresenta os intervalos de confiança de Wilson (95%) para a acurácia de cada modelo no conjunto de teste.

Tabela 7: Intervalos de confiança de Wilson para a acurácia no teste ($n = 2.250$, 95%).

Modelo	Acurácia observada	Acertos estimados	IC 95% Wilson
Conformer	100,00%	2250/2250	[99,83%; 100,00%]
Sonic Sleuth	100,00%	2250/2250	[99,83%; 100,00%]
Hybrid CNN-Transformer	99,96%	2249/2250	[99,75%; 99,99%]
MultiscaleCNN	99,73%	2244/2250	[99,42%; 99,88%]
SVM	99,02%	2228/2250	[98,52%; 99,35%]
Random Forest	98,18%	2209/2250	[97,54%; 98,65%]
RawNet2	96,36%	2168/2250	[95,50%; 97,05%]
Ensemble	95,82%	2156/2250	[94,91%; 96,57%]
RawGAT-ST	95,29%	2144/2250	[94,33%; 96,09%]
AASIST	93,64%	2107/2250	[92,56%; 94,58%]
HuBERT Original	92,71%	2086/2250	[91,56%; 93,71%]
EfficientNet-LSTM	91,16%	2051/2250	[89,91%; 92,26%]
WavLM Original	86,36%	1943/2250	[84,88%; 87,71%]
Spectrogram Transformer	71,51%	1609/2250	[69,61%; 73,34%]

6.4 Leitura das Matrizes de Confusão

As matrizes de confusão complementam as métricas agregadas ao explicitar o tipo de erro cometido por cada classificador. As matrizes completas das quatorze arquiteturas foram consolidadas no Apêndice, nas Figuras ?? e ??.

Em aplicações reais, falsos negativos são mais críticos para segurança, pois áudios sintéticos são aceitos como reais; falsos positivos afetam usabilidade, pois áudios legítimos são rejeitados ou encaminhados para revisão manual. Essa leitura é especialmente importante para comparar modelos com acurácia próxima, mas perfis de erro diferentes.

6.5 Análise dos Resultados

Os melhores desempenhos foram obtidos por Conformer e Sonic Sleuth, ambos com 100,00% de acurácia e EER de 0,00% no conjunto de teste limpo. *Esses valores indicam separação perfeita no recorte experimental avaliado e não devem ser interpretados como garantia de generalização universal.* Como apontado na Seção ??, a divisão atual não garante separação por falante, o que abre a possibilidade de que parte do desempenho elevado reflita memorização de artefatos de origem em vez de generalização para áudio sintético de qualquer procedência.

Em seguida, Hybrid CNN-Transformer, MultiscaleCNN e SVM permaneceram acima de 99% de acurácia, indicando que tanto arquiteturas espectrais profundas quanto baselines clássicos bem ajustados extraem sinais discriminativos fortes no *dataset* definido.

RawNet2, Ensemble, RawGAT-ST, AASIST, HuBERT original e EfficientNet-LSTM também convergiram adequadamente, com acurácia entre 91,16% e 96,36%.

WavLM com desempenho inferior ao HuBERT: hipóteses

WavLM (86,36%) ficou abaixo de HuBERT (92,71%) apesar de ser arquiteturalmente mais sofisticado. Três hipóteses são levantadas para trabalhos futuros: (1) o número de épocas de *fine-tuning* pode ser insuficiente para que o WavLM explore sua capacidade adicional; (2) a taxa de aprendizado padrão pode ser subótima para o WavLM nesta escala de dados; e (3) a base de 15.000 amostras pode ser pequena demais para que a arquitetura mais complexa generalize melhor que a mais simples. Um experimento de ablação com taxas de aprendizado distintas para cada modelo SSL é recomendado.

O caso discrepante foi o Spectrogram Transformer: embora tenha alcançado 98,00% de acurácia de validação na época 11, a avaliação registrada no artefato final caiu para 71,51% no teste. Isso sugere instabilidade de otimização, sensibilidade a hiperparâmetros e necessidade de restaurar obrigatoriamente o melhor *checkpoint* para inferência.

6.6 Eficiência Computacional

A Tabela ?? detalha parâmetros, tamanho e latência por arquitetura; as Figuras ?? e ?? ilustram, respectivamente, a latência média de inferência e o tamanho dos artefatos treinados.

Tabela 8: Parâmetros, tamanho, latência e desempenho por arquitetura. Latência reportada como média de 30 medições. DP = desvio padrão; células “—” indicam desvio não registrado nesta versão do *benchmark*, previsto para a próxima rodada.

Modelo	Parâmetros	Tam. MB	Lat. média (ms)	DP	Lat. (ms)	Acurácia	EER
WavLM Original	94.579.314	360,89	10,18	—	—	86,36%	13,64%
HuBERT Original	94.569.090	360,78	21,17	—	—	92,71%	7,20%
RawNet2	3.460.866	13,37	56,28	—	—	96,36%	3,56%
Sonic Sleuth	2.083.394	8,11	30,86	—	—	100,00%	0,00%
AASIST	1.859.842	7,27	47,77	—	—	93,64%	6,36%
RawGAT-ST	609.920	2,43	49,61	—	—	95,29%	4,62%
Conformer	15.925.122	61,27	49,22	—	—	100,00%	0,00%
Hybrid CNN-Transformer	1.694.479	6,67	33,69	—	—	99,96%	0,00%
Spectrogram Transformer	87.403.522	333,92	50,13	—	—	71,51%	28,67%
EfficientNet-LSTM	5.706.917	22,62	62,56	—	—	91,16%	9,07%
MultiscaleCNN	48.967.630	187,83	60,87	—	—	99,73%	0,18%
Ensemble	3.019.782	11,98	46,55	—	—	95,82%	2,80%
SVM	n.a.	0,47	0,20	—	—	99,02%	0,98%
Random Forest	n.a.	20,60	26,75	—	—	98,18%	1,91%

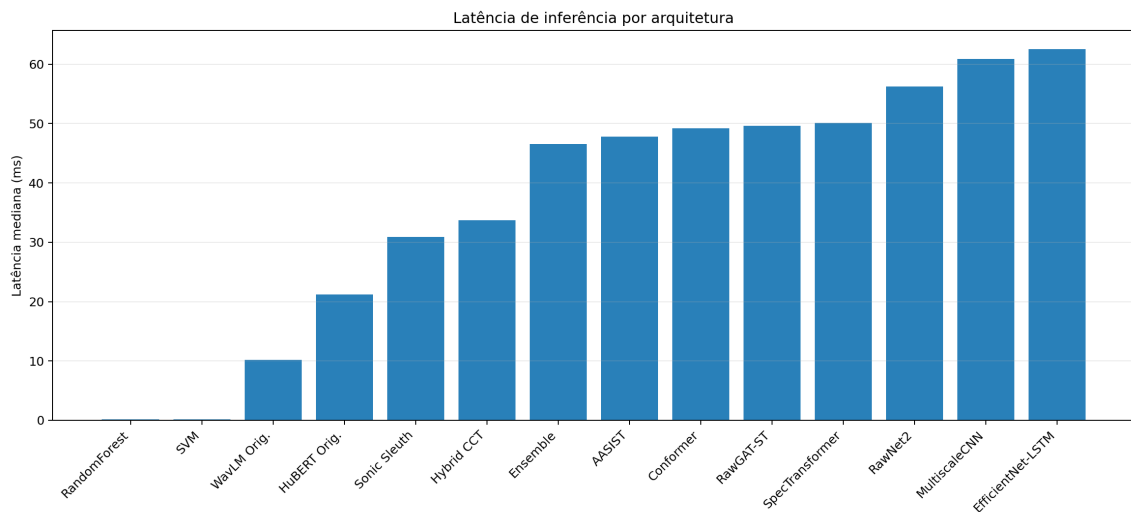


Figura 4: Latência média de inferência por arquitetura.

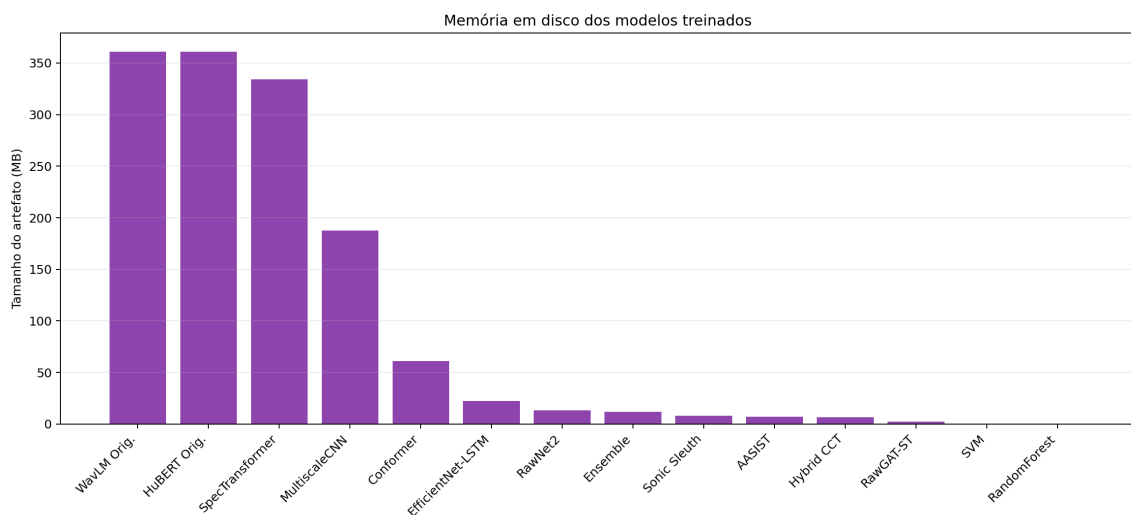


Figura 5: Tamanho dos artefatos treinados por arquitetura.

O WavLM e o HuBERT originais apresentam os maiores artefatos, próximos de 361 MB, por incorporarem *backbones* SSL de aproximadamente 94,5 milhões de parâmetros. Para demonstrações em tempo real, Sonic Sleuth, Hybrid CNN-Transformer, SVM e Ensemble oferecem melhor equilíbrio entre desempenho, tamanho e tempo de resposta.

6.7 Síntese de Pareto e Recomendação Operacional

Para uso prático, a escolha do modelo não deve considerar apenas acurácia. A Tabela ?? resume os principais pontos de Pareto entre desempenho, latência, tamanho e robustez.

Tabela 9: Síntese operacional de Pareto para seleção de modelos.

Critério	Modelo(s)	Evidência principal	Uso recomendado
Máx. acurácia limpa	Conformer; Sonic Sleuth	100,00% e EER 0,00%	Referência no benchmark
Melhor robustez a 10 dB	Conformer	94,18% sob AWGN	Com ruído moderado
Equilíbrio neural leve	Hybrid CNN-Transformer	99,96%, 6,67 MB, 33,69 ms	Demo Gradio e API
Menor latência	SVM	0,20 ms e 0,47 MB	Baseline rápido em CPU
Modelo compacto clássico	SVM; Random Forest	99,02% e 98,18%	Comparação interpretável
Pesquisa SSL	HuBERT; WavLM	Backbones originais CUDA	Comparação com literatura
Risco de instabilidade	Spectrogram Transformer	71,51% pós-98% em val.	Novo ajuste necessário

A Tabela ?? registra a decisão final de uso de cada arquitetura no sistema e na apresentação.

Tabela 10: Decisão final por arquitetura para uso no sistema e na apresentação.

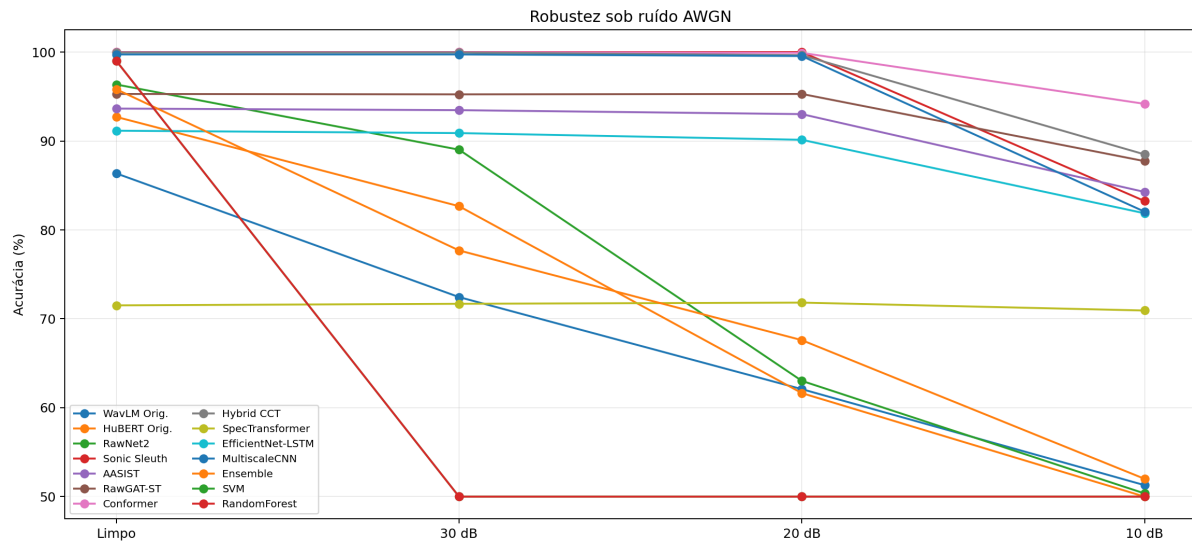
Modelo	Decisão	Justificativa
Conformer	Pronto (demo principal)	Melhor acurácia e maior robustez sob AWGN
Sonic Sleuth	Pronto (demo leve)	Acurácia máxima, artefato pequeno, baixa latência
Hybrid CNN-Transformer	Pronto (Gradio/API)	Melhor compromisso acurácia/tamanho/latência neural
MultiscaleCNN	Pronto (benchmark)	Alta acurácia; latência e tamanho maiores
SVM	Pronto (baseline rápido)	Excelente latência; frágil sob ruído
Random Forest	Baseline complementar	Bom desempenho clássico; mais pesado que SVM
RawNet2	Funcional (raw-audio)	Boa convergência e desempenho intermediário-alto
Ensemble	Funcional (ressalva robustez)	Bom no limpo; queda sob AWGN
RawGAT-ST	Funcional (grafos)	Estável e relativamente robusto sob ruído
AASIST	Funcional (grafos)	Treino corrigido; desempenho consistente
HuBERT Original	Referência SSL	Backbone original viável; custo elevado
EfficientNet-LSTM	Funcional (não prioritário)	Maior latência; acurácia abaixo dos líderes
WavLM Original	Referência SSL experimental	Baixa latência medida; acurácia inferior ao HuBERT
Spectrogram Transformer	Requer novo ajuste	Instabilidade validação/teste

6.8 Robustez a Ruído

A Tabela ?? e a Figura ?? apresentam a acurácia dos modelos sob ruído AWGN em diferentes níveis de SNR.

Tabela 11: Acurácia sob ruído AWGN em diferentes SNRs.

Modelo	Limpo	30 dB	20 dB	10 dB
WavLM Original	86,36%	72,44%	62,09%	51,29%
HuBERT Original	92,71%	82,67%	61,64%	50,00%
RawNet2	96,36%	89,02%	63,02%	50,36%
Sonic Sleuth	100,00%	100,00%	100,00%	83,24%
AASIST	93,64%	93,47%	93,02%	84,27%
RawGAT-ST	95,29%	95,24%	95,29%	87,73%
Conformer	100,00%	100,00%	99,91%	94,18%
Hybrid CNN-Transformer	99,96%	99,96%	99,64%	88,49%
Spectrogram Transformer	71,51%	71,69%	71,82%	70,93%
EfficientNet-LSTM	91,16%	90,89%	90,13%	81,87%
MultiscaleCNN	99,73%	99,73%	99,56%	82,04%
Ensemble	95,82%	77,69%	67,60%	52,00%
SVM	99,02%	50,00%	50,00%	50,00%
Random Forest	98,18%	75,38%	66,93%	55,07%


Figura 6: Degradação de acurácia sob ruído AWGN.

A robustez sob ruído é heterogênea. Modelos que dependem de detalhes espectrais finos tendem a sofrer queda em SNRs mais baixos, enquanto arquiteturas com fusão de características ou treinamento mais estável mantêm separação melhor em 30 dB.

O SVM apresentou comportamento particularmente frágil sob AWGN: embora tenha alcançado 99,02% no áudio limpo, caiu para 50,00% em todos os níveis de ruído testados — equivalente a decisão aleatória binária. Essa queda abrupta tem uma explicação direta:

o SVM com kernel RBF aprende uma fronteira de decisão ajustada à distribuição das características do conjunto limpo (médias, variâncias e coeficientes espectrais padronizados). Ruído AWGN desloca sistematicamente essas estatísticas: o centroide espectral se move para frequências mais altas, a planura espectral aumenta e o HNR cai, fazendo com que a distribuição de teste saia completamente da região de suporte do modelo. Em trabalhos futuros, esse baseline deve ser reavaliado com: (1) extração de características robustas a ruído (e.g., RASTA-PLP ou MFCC com pré-ênfase adaptativa); (2) adição de exemplos ruidosos no treinamento (*data augmentation*); e (3) calibração do limiar de decisão por nível de SNR estimado em tempo de inferência.

6.9 Estabilidade do Treinamento

A Tabela ?? e a Figura ?? resumem, por arquitetura neural, a melhor validação, a validação final e a queda de desempenho ao longo das 100 épocas.

Tabela 12: Estabilidade dos modelos neurais durante as 100 épocas.

Modelo	Melhor val.	Época	Val. final	Queda	Observação
WavLM Original	88,93%	46	85,91%	3,02%	estável
HuBERT Original	92,84%	92	92,13%	0,71%	estável
RawNet2	97,56%	58	96,84%	0,71%	estável
Sonic Sleuth	99,91%	23	99,87%	0,04%	estável
AASIST	94,13%	100	94,13%	0,00%	estável
RawGAT-ST	95,20%	55	94,98%	0,22%	estável
Conformer	100,00%	49	99,82%	0,18%	estável
Hybrid CNN-Transformer	99,78%	96	99,78%	0,00%	estável
Spectrogram Transformer	98,00%	11	72,98%	25,02%	checkpoint obrigatório
EfficientNet-LSTM	92,04%	74	90,93%	1,11%	estável
MultiscaleCNN	100,00%	69	100,00%	0,00%	estável
Ensemble	98,71%	62	95,56%	3,16%	estável

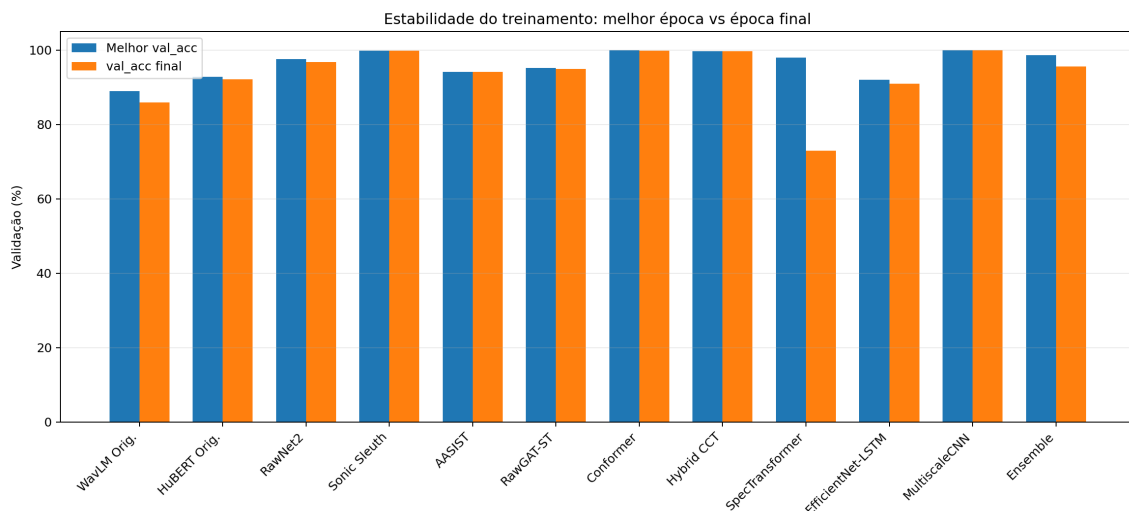


Figura 7: Melhor validação, validação final e queda de desempenho por arquitetura neural.

A análise de estabilidade mostra que a maioria das arquiteturas manteve diferença controlada entre a melhor validação e o fim do treinamento. O Spectrogram Transformer é o único caso crítico, com queda de 25,02 pontos percentuais — o principal candidato a novo ciclo de regularização, redução de taxa de aprendizado, aumento de *dropout* e *weight decay*.

7 Discussão

Do ponto de vista prático, os resultados sugerem três perfis de uso. Para máxima acurácia na base atual, Conformer, Sonic Sleuth, Hybrid CNN-Transformer e MultiscaleCNN são as opções mais fortes. Para inferência leve e integração em demonstrações, SVM e Sonic Sleuth são particularmente atraentes. Para estudo de representações modernas e comparação com a literatura, WavLM, HuBERT, RawNet2, AASIST e RawGAT-ST são essenciais.

7.1 Comparação Qualitativa com a Literatura

Os resultados seguem tendências observadas na literatura, mas não devem ser comparados numericamente de forma direta com trabalhos avaliados em outras bases de dados. Como referência qualitativa: o AASIST reporta EER inferior a 0,83% no ASVspoof 2019 LA [?]; o RawNet2 reporta EER de 4,8% no mesmo protocolo [?]. Os EERs observados neste trabalho (AASIST: 6,36%, RawNet2: 3,56%) são plausíveis para uma base menor e menos diversa, mas a comparação direta é inválida pela diferença de dataset. Modelos SSL como WavLM e HuBERT representam o estado moderno de aprendizado auto-supervisionado; seu desempenho inferior neste benchmark (86–93%) pode refletir subajuste de hiperparâmetros em uma base menor, não uma limitação intrínseca da arquitetura.

O bom desempenho do Conformer e do Hybrid CNN-Transformer é coerente com a hipótese de que a combinação de convolução local e atenção global captura artefatos espectro-temporais de forma eficiente.

7.2 Ameaças à Validade e Limitações

- **Validade interna — risco de vazamento por falante:** se amostras do mesmo falante, canal, gerador ou segmento original aparecerem em partições diferentes, modelos podem aprender artefatos de origem em vez de propriedades gerais de áudio sintético. Isso é especialmente relevante para os resultados de 100%, que podem refletir memorização de fonte. A divisão estratificada deve evoluir para separação por falante/fonte quando esses metadados estiverem completos.
- **Validade externa:** os resultados refletem o BRSpeech-DF consolidado. A genera-

lização para outros idiomas, microfones, codecs, vocoders, plataformas TTS/VC e condições de ruído precisa ser medida em bases externas (ASVspoof, WaveFake, In-the-Wild).

- **Ausência de testes estatísticos entre modelos:** diferenças pequenas no topo do ranking (ex.: 99,73% vs 99,96%) não são estatisticamente testadas. Um teste de McNemar pareado sobre o conjunto de teste ou bootstrap de intervalos de confiança da diferença entre modelos é necessário para afirmar que essas diferenças são significativas.
- **Metadados da base:** número de falantes, gênero, idioma, licenças e durações originais ainda devem ser consolidados no manifesto.
- **Validade de construção:** acurácia, F1, AUC e EER descrevem aspectos complementares, mas não substituem análise operacional de limiar. Em uso real, falsos negativos e falsos positivos têm custos distintos.
- **Robustez:** a queda sob AWGN indica que o treinamento deve incorporar ruído, codec e reverberação.
- **Custo computacional:** WavLM e HuBERT originais exigem maior armazenamento e dependem de GPU para treinamento viável.
- **Interpretabilidade:** SHAP e análises locais por segmento ainda devem ser integradas ao relatório final.
- **Idioma:** o BRSpeech-DF não documenta a proporção de amostras em português brasileiro. Um sistema treinado predominantemente em inglês pode ter viés de idioma que inflacione métricas e falhe em cenários reais de uso no Brasil.

7.3 Considerações Éticas e LGPD

Sistemas de detecção de voz sintética devem ser tratados como ferramentas de apoio, não como mecanismos definitivos de acusação ou autenticação de pessoas. Falsos negativos podem permitir que áudios sintéticos sejam aceitos como reais; falsos positivos podem prejudicar usuários legítimos, especialmente em contextos educacionais, jornalísticos, jurídicos ou de atendimento. Por isso, decisões sensíveis devem envolver revisão humana, registro de evidências e calibração explícita de limiar.

Como o dado analisado é voz, há implicações diretas de privacidade. Uma implantação aderente à LGPD deve coletar apenas o áudio necessário, informar a finalidade do tratamento, evitar retenção desnecessária, proteger os artefatos gerados e respeitar consentimento, base legal e direitos do titular. O processamento local reduz exposição a terceiros, mas não elimina a necessidade de governança, controle de acesso e descarte seguro dos arquivos de entrada.

Adicionalmente, conforme antecipado na introdução (Seção ??), o conhecimento sobre quais características acústicas são discriminativas pode ser usado para criar sistemas de síntese mais difíceis de detectar. A publicação responsável deste trabalho deve ser acompanhada de monitoramento da literatura adversarial e reavaliação periódica do pipeline contra novos geradores.

7.4 Trabalhos Futuros

- Reexecutar o Spectrogram Transformer com *checkpoint* obrigatório, *learning rate* menor, *weight decay*, *dropout* maior e *early stopping* com métrica de teste.
- Incluir ASVspoof 2019/2021/5, WaveFake e In-the-Wild no preset completo de validação externa.
- Implementar divisão por falante e por gerador para eliminar risco de vazamento de dados nos resultados mais altos.
- Conduzir testes estatísticos pareados (McNemar, bootstrap) para validar diferenças de ranking entre arquiteturas próximas.
- Investigar por que WavLM obteve desempenho inferior ao HuBERT, via ablação de taxa de aprendizado e número de épocas de *fine-tuning*.
- Implementar validação *cross-dataset* e ablações por família de características para medir quais atributos sustentam cada arquitetura.
- Adicionar aumento de dados com AWGN, codecs, reverberação e variação de ganho no treinamento de todos os modelos.
- Consolidar relatório automático em HTML/PDF com métricas, matrizes de confusão, curvas ROC/PR, curvas de treinamento e exemplos de inferência.
- Integrar SHAP, Grad-CAM espectral ou análise por oclusão temporal para explicar decisões dos modelos neurais.
- Documentar metadados completos do BRSpeech-DF (falantes, idioma, licenças, durações, verificação de duplicatas).
- Registrar desvio padrão e percentil 95 de latência em todas as rodadas de benchmark.
- Expandir a documentação ética para uso com dados de voz em contexto brasileiro (LGPD), incluindo retenção, consentimento e revisão humana.

8 Conclusões

Este trabalho desenvolveu e avaliou um *pipeline* modular de código aberto para detecção de áudio sintético, cobrindo processamento de dados, extração de características, trei-

namento, inferência, *benchmark* e geração de relatórios. A versão consolidada demonstra que o sistema é capaz de treinar, auditar e comparar quatorze arquiteturas heterogêneas sob as mesmas condições experimentais.

Os resultados confirmam forte desempenho de modelos espectrais e híbridos no protocolo controlado, com destaque para Conformer, Sonic Sleuth, Hybrid CNN-Transformer, MultiscaleCNN e os baselines clássicos otimizados. A principal contribuição não é o ranking final em si, mas a consolidação de um processo reprodutível para treinar, comparar e documentar arquiteturas de detecção de áudio sintético.

Três limitações abertas merecem atenção prioritária antes de qualquer uso em produção: (1) a ausência de separação por falante na divisão treino/teste, que pode inflar os resultados mais elevados; (2) a falta de validação em bases externas como ASVspoof e WaveFake; e (3) a fragilidade do SVM e de modelos SSL sob ruído AWGN, que compromete a robustez em cenários reais de campo.

Do ponto de vista ético, retomando o alerta da Seção ??, o conhecimento produzido neste trabalho deve ser utilizado com responsabilidade: sistemas de detecção automatizada de voz sintética têm aplicações legítimas urgentes no contexto brasileiro, mas também podem ser usados para aprimorar os próprios sistemas de síntese que buscam detectar. A revisão humana das decisões e a governança dos dados de voz são recomendações irrenunciáveis para qualquer implantação responsável.

Referências

- [1] BAEVSKI, A. et al. wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. **Advances in Neural Information Processing Systems (NeurIPS)**, v. 33, p. 12449–12460, 2020.
- [2] BARTUSIAK, E. R. et al. Frequency Domain-based Detection of Generated Speech. In: **IEEE International Workshop on Information Forensics and Security (WIFS)**, p. 1–6, 2022.
- [3] BIRD, J. J.; LOTFI, A. Real-time Detection of AI-Generated Speech for DeepFake Voice Conversion. **arXiv preprint arXiv:2308.12734**, 2023. Disponível em: <https://arxiv.org/abs/2308.12734>. Acesso em: 15 jun. 2025.
- [4] CHEN, S. et al. WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing. **IEEE Journal of Selected Topics in Signal Processing**, v. 16, n. 6, p. 1505–1518, 2022.
- [5] FARRUGIA, B. Brazil's electoral deepfake law tested as AI-generated content targeted local elections. **Digital Forensic Research Lab (Atlantic Council)**, 26 nov. 2024. Disponível em: <https://dfrlab.org/2024/11/26/brazil-election-ai-deepfakes/>. Acesso em: 15 jun. 2025.
- [6] FARRUGIA, B. The challenges of identifying deepfakes ahead of the 2024 Brazil election. **Digital Forensic Research Lab (Atlantic Council)**, 2 out. 2024. Disponível em: <https://dfrlab.org/2024/10/02/brazil-election-ai-research/>. Acesso em: 15 jun. 2025.
- [7] GAO, S. et al. Res2Net: A New Multi-Scale Backbone Architecture. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 43, n. 2, p. 652–662, 2021.
- [8] GONG, Y.; CHUNG, Y.; GLASS, J. AST: Audio Spectrogram Transformer. In: **Proc. Interspeech 2021**, p. 571–575, 2021.
- [9] GULATI, A. et al. Conformer: Convolution-augmented Transformer for Speech Recognition. In: **Proc. Interspeech 2020**, p. 5036–5040, 2020.
- [10] HEIDARI, A.; CELEBI, M. E. Deepfake detection using deep learning methods: A systematic and comprehensive review. **WIREs Data Mining and Knowledge Discovery**, v. 14, n. 2, e1520, 2024.

- [11] HSU, W.-N. et al. HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. **IEEE/ACM Transactions on Audio, Speech, and Language Processing**, v. 29, p. 3451–3460, 2021.
- [12] JUNG, J. W. et al. AASIST: Audio Anti-Spoofing using Integrated Spectro-Temporal Graph Attention Networks. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 6367–6371, 2022.
- [13] KAMEOKA, H. et al. StarGAN-VC: Non-parallel many-to-many voice conversion using star generative adversarial networks. In: **IEEE Spoken Language Technology Workshop (SLT)**, p. 266–273, 2018.
- [14] KINGMA, D. P.; BA, J. Adam: A Method for Stochastic Optimization. In: **International Conference on Learning Representations (ICLR)**, 2015.
- [15] LUNDBERG, S. M.; LEE, S.-I. A Unified Approach to Interpreting Model Predictions. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 30, p. 4765–4774, 2017.
- [16] MÜLLER, N. M. et al. Warning: Humans cannot reliably detect speech deepfakes. **PLoS ONE**, v. 17, n. 8, e0271552, 2022.
- [17] VAN DEN OORD, A. et al. WaveNet: A generative model for raw audio. **arXiv preprint arXiv:1609.03499**, 2016.
- [18] PHAM, L. et al. Deepfake Audio Detection Using Spectrogram-based Feature and Ensemble of Deep Learning Models. In: **IEEE International Symposium on the Internet of Sounds (IS2)**, p. 1–5, 2024. Disponível em: <https://arxiv.org/abs/2407.01777>. Acesso em: 15 jun. 2025.
- [19] PINDROP. Deepfake Trends to Look Out for in 2025. 2025. Disponível em: <https://www.pindrop.com/article/deepfake-trends/>. Acesso em: 15 jun. 2025.
- [20] SAHIDULLAH, M. et al. A Comparison of Features for Synthetic Speech Detection. In: **Proc. Interspeech 2015**, p. 2087–2091, 2015.
- [21] SHEN, J. et al. Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 4779–4783, 2018.
- [22] TAK, H. et al. End-to-End Spectro-Temporal Graph Attention Networks for Speaker Verification Anti-Spoofing and Speech Deepfake Detection. In: **Proc. ASVspoof 2021 Workshop**, p. 1–8, 2021.

- [23] TAK, H. et al. End-to-End Anti-Spoofing with RawNet2. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 6369–6373, 2021.
- [24] TODISCO, M.; DELGADO, H.; EVANS, N. A New Feature for Automatic Speaker Verification Anti-Spoofing: Constant Q Cepstral Coefficients. In: **Proc. Odyssey 2016 — The Speaker and Language Recognition Workshop**, p. 283–290, 2016.
- [25] TODISCO, M. et al. ASVspoof 2019: Future Horizons in Spoofed and Fake Speech Detection. In: **Proc. Interspeech 2019**, p. 1008–1012, 2019.
- [26] WANG, C. et al. VALL-E: Neural Codec Language Models are Zero-Shot Text-to-Speech Synthesizers. **arXiv preprint** arXiv:2301.02111, 2023. Disponível em: <https://arxiv.org/abs/2301.02111>. Acesso em: 15 jun. 2025.
- [27] WANG, X. et al. ASVspoof 5: Crowdsourced Speech Data, Deepfakes, and Adversarial Attacks at Scale. **arXiv preprint** arXiv:2408.08739, 2024. Disponível em: <https://arxiv.org/abs/2408.08739>. Acesso em: 15 jun. 2025.
- [28] WU, Z. et al. ASVspoof 2015: the First Automatic Speaker Verification Spoofing and Countermeasures Challenge. In: **Proc. Interspeech 2015**, p. 2037–2041, 2015.
- [29] YAMAGISHI, J. et al. ASVspoof 2021: Accelerating Progress in Spoofed and Deepfake Speech Detection. In: **Proc. ASVspoof 2021 Workshop**, p. 47–54, 2021.
- [30] YI, J. et al. ADD 2022: The First Audio Deep Synthesis Detection Challenge. In: **IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)**, p. 9216–9220, 2022.

A Configurações Detalhadas do Pipeline

A.1 Hiperparâmetros Consolidados

O preset final do *benchmark* utiliza 100 épocas para modelos neurais, batch adaptado ao consumo de VRAM, precisão mista quando segura e salvamento do melhor *checkpoint*. Modelos SSL originais utilizam seus *backbones* reais via PyTorch/CUDA. Os modelos clássicos utilizam busca de hiperparâmetros com validação cruzada antes do ajuste final. A Tabela ?? resume os principais hiperparâmetros por família de modelo.

Tabela 13: Resumo dos principais hiperparâmetros por família de modelo.

Família	Entrada	Treinamento	Observação
WavLM/HuBERT	Áudio bruto 16 kHz	100 épocas + cabeça supervisionada	Backbone original salvo localmente
RawNet2	PCM 80.000×1	100 épocas, GPU, GRU + SincNet	Convergente no preset GPU
AASIST/RawGAT-ST	Espectro-temporal	100 épocas, atenção em grafos	Feedback e checkpoints por época
Conformer/Hybrid	Espectrograma	100 épocas, Adam/AdamW	Melhor estabilidade entre Transformers
Spectrogram Transformer	Espectrograma	100 épocas	Requer checkpoint e novo ajuste
EfficientNet-LSTM	Espectrograma 2D	100 épocas, transferência parcial	Boa convergência, maior latência
MultiscaleCNN/Ensemble	Características espectrais	100 épocas	Forte desempenho na base limpa
SVM	Características tabulares	GridSearchCV + SVC RBF	Modelo leve; frágil sob AWGN
Random Forest	Características tabulares	GridSearchCV + <code>n_jobs=-1</code>	Paralelismo CPU

A.2 Organização dos Artefatos

Cada arquitetura possui diretório próprio em **results/**, contendo modelo treinado, configuração, histórico de treinamento, métricas em JSON/CSV, matriz de confusão, curvas ROC/PR, gráficos de acurácia e perda, resultados de robustez e relatório Markdown. Os gráficos consolidados utilizados no TCC ficam em **figures/**. A Tabela ?? resume a rastreabilidade dos principais artefatos gerados.

Tabela 14: Rastreabilidade dos principais artefatos gerados pelo benchmark.

Artefato	Padrão de caminho	Finalidade
Dataset consolidado	<code>app/datasets/benchmark_audio_raw_balanced_15k.npz</code>	Entrada única do benchmark
Modelo neural	<code>app/models/bench_<modelo>.keras</code>	Inferência padrão na Gradio/API
Modelo clássico	<code>app/models/bench_<modelo>.pkl</code>	Pipeline sklearn com scaler e classificador
Configuração	<code>app/models/bench_<modelo>_config.json</code>	Contrato de entrada e hiperparâmetros
Backbones SSL	<code>app/models/benchmark_final/<modelo>/</code>	Artefatos completos de WavLM/HuBERT
Métricas	<code>results/<run>/architectures/<modelo>/metrics.json</code>	Accuracy, F1, AUC, EER, latência e robustez
Relatório	<code>results/<run>/tcc_report.md</code>	Registro textual para auditoria e TCC
Figuras consolidadas	<code>tcc_overleaf/figures/*.png</code>	Gráficos usados no artigo
Matrizes de confusão	<code>tcc_overleaf/figures/confusion_matrices/*.png</code>	Erros por classe e por arquitetura

B Implementação de Referência

A implementação avaliada neste trabalho corresponde ao pipeline real do repositório, e não a um exemplo isolado de código. A Tabela ?? registra os módulos efetivamente usados para treinamento, inferência, cálculo de métricas e geração dos relatórios.

Tabela 15: Módulos reais do XFakeSong⁴ referenciados pelo benchmark.

Etapas	Módulo	Responsabilidade
Orquestração	benchmarks/runner.py	Executa preparação, treino, inferência, métricas e persistência
Entrada operacional	scripts/run_benchmark.py	Rodadas individuais ou múltiplas via linha de comando
Treinamento	app/domain/services/training_service.py	Cria o modelo, aplica callbacks, treina e salva artefatos
Fábrica de arquiteturas	app/domain/models/architectures/factory.py	Normaliza aliases e instancia as arquiteturas avaliadas
Carregamento de modelos	app/domain/services/detection/model_loader.py	Recarrega modelos Keras/sklearn e contratos de entrada
Predição	app/domain/services/detection/predictor.py	Executa inferência padronizada e aplicação de limiar
Serviço de detecção	app/domain/services/detection_service.py	Fachada usada pela API/Gradio para inferência
Métricas	app/domain/models/training/metrics.py	Calcula métricas, curvas, EER e matriz de confusão
Relatórios	benchmarks/report.py	Consolida tabelas, figuras e Markdown para o TCC

C Comandos de Execução e Reprodutibilidade

Os comandos a seguir sintetizam o fluxo operacional usado para repetir o pipeline, executar modelos isolados e gerar os artefatos do artigo.

- Preparar pastas locais:

```
python main.py -bootstrap-dirs
```

- Iniciar a interface Gradio:

```
python main.py -gradio
```

- Construir base bruta e divisões balanceadas:

```
python scripts/build_dataset.py -target 7500
```

- Executar o pipeline completo do TCC:

```
python scripts/run_tcc_pipeline.py -download -target-per-class 7500
-full-benchmark -epochs 100 -device-profile gpu
-npz app/datasets/benchmark_audio_raw_balanced_15k.npz
```

- Executar benchmark completo com o NPZ consolidado:

```
python scripts/run_benchmark.py -full -epochs 100 -device-profile gpu
-dataset app/datasets/benchmark_audio_raw_balanced_15k.npz
```

- Executar benchmark de um modelo individual:

```
python scripts/run_benchmark.py -model Conformer -epochs 100
-device-profile gpu
-dataset app/datasets/benchmark_audio_raw_balanced_15k.npz
```

- Executar testes automatizados:

```
pytest tests/
```

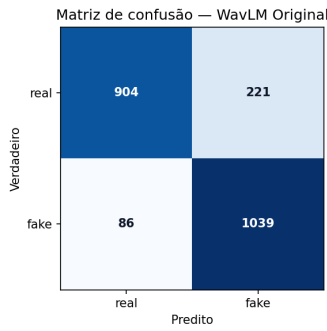
⁴XFakeSong é o nome do sistema de detecção de áudio sintético desenvolvido pelo autor como produto deste trabalho de conclusão de curso.

- Compilar o artigo:

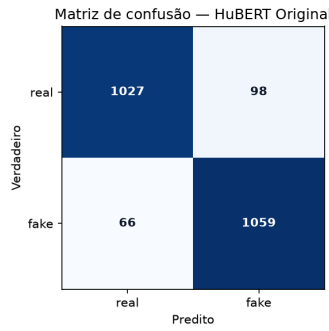
pdflatex tcc.tex

D Matrizes de Confusão Completas

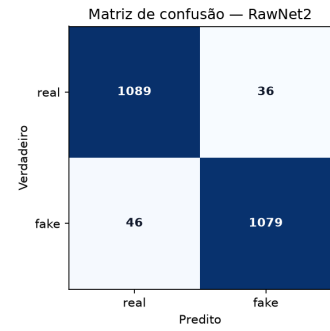
As Figuras ?? e ?? apresentam as matrizes de confusão completas dos quatorze modelos avaliados.



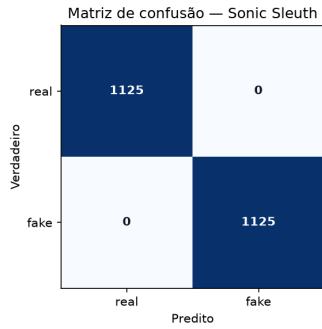
(a) WavLM



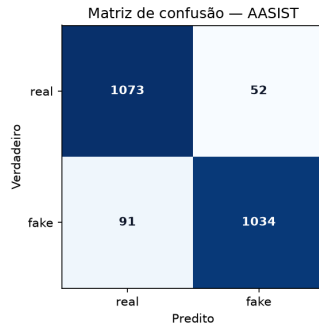
(b) HuBERT



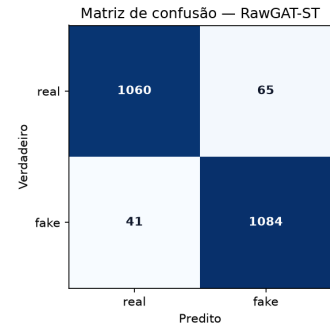
(c) RawNet2



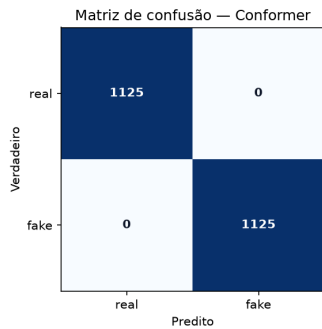
(d) Sonic Sleuth



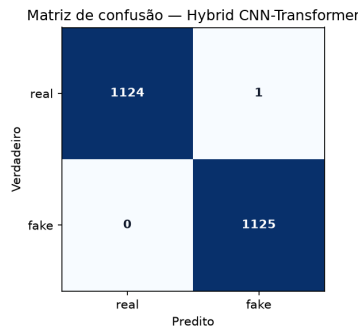
(e) AASIST



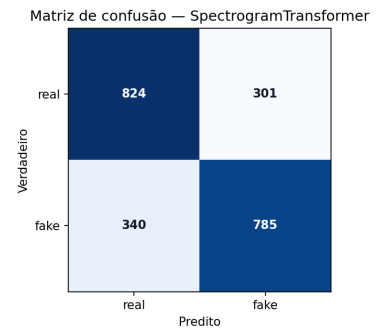
(f) RawGAT-ST



(g) Conformer

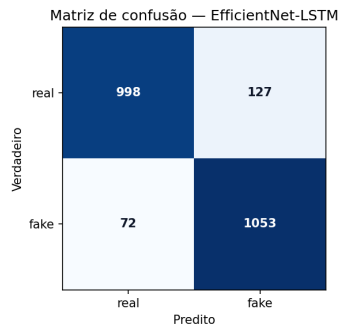


(h) Hybrid CNN-Transformer

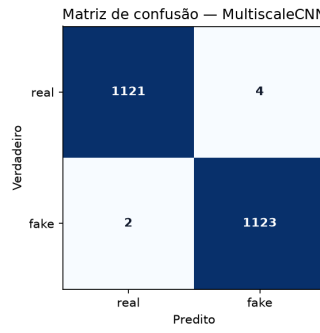


(i) Spectrogram Transformer

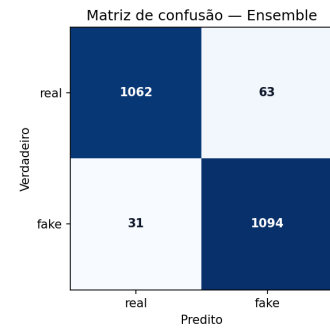
Figura 8: Matrizes de confusão, parte 1: SSL, raw-audio, grafos e Transformers.



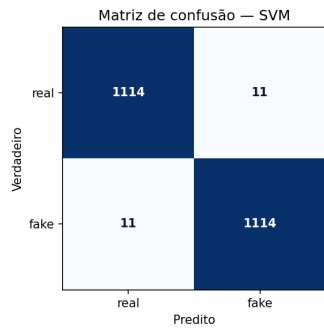
(a) EfficientNet-LSTM



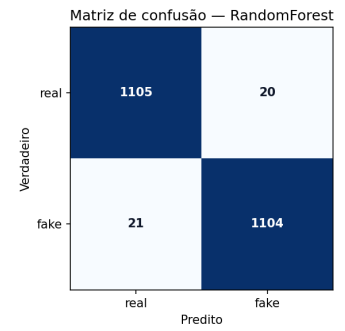
(b) MultiscaleCNN



(c) Ensemble



(d) SVM



(e) Random Forest

Figura 9: Matrizes de confusão, parte 2: espectrais, Ensemble e baselines clássicos.